

Tutorial: Nhận diện Ngôn ngữ Ký hiệu (Olympic AI)

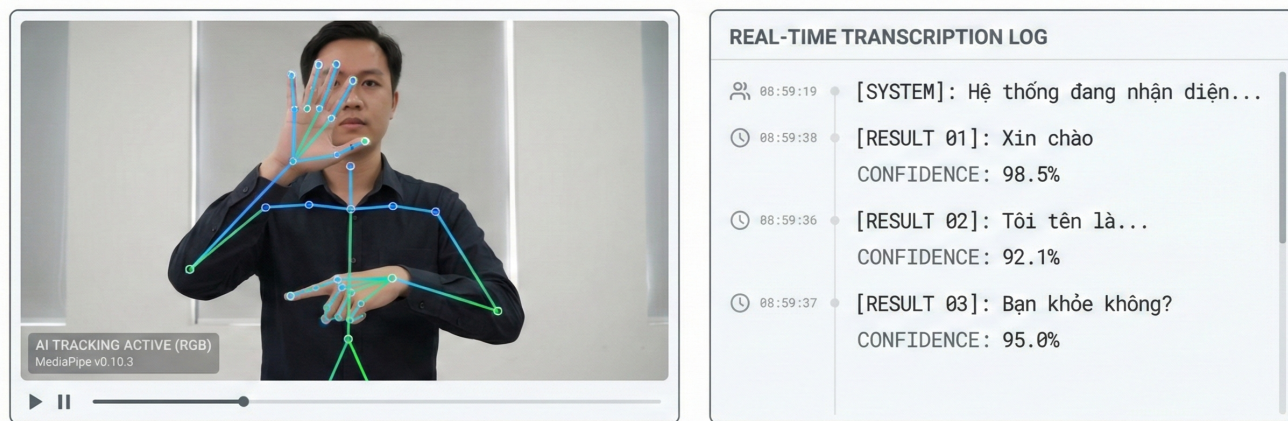
Nguyễn Phúc Thịnh
Trần Hoàng Duy

Nguyễn Quốc Thái
Đinh Quang Vinh

I. Bối cảnh bài toán

Ngôn ngữ ký hiệu (Sign Language) là phương tiện giao tiếp tự nhiên và quan trọng nhất của cộng đồng người khiếm thính. Tuy nhiên, đa số người nghe nói bình thường không được trang bị kiến thức về ngôn ngữ này, tạo nên một rào cản giao tiếp lớn, khiến người khiếm thính gặp nhiều khó khăn trong việc tiếp cận thông tin, giáo dục, y tế và các dịch vụ công. Trong nhiều bối cảnh, việc chuyển đổi ngôn ngữ ký hiệu sang văn bản hoặc giọng nói vẫn chủ yếu dựa vào đội ngũ phiên dịch viên, vốn khan hiếm và tốn kém.

Những năm gần đây, sự phát triển mạnh mẽ của Thị giác máy tính (Computer Vision) và Học sâu (Deep Learning) đã mở ra các hướng tiếp cận mới nhằm thu hẹp khoảng cách này. Các kỹ thuật như ước lượng dáng điệu (Pose Estimation) cho phép máy tính trích xuất chính xác các điểm đặc trưng trên cơ thể, bàn tay và khuôn mặt từ dữ liệu video mà không cần đến các thiết bị đeo hỗ trợ, từ đó tạo tiền đề cho việc xây dựng các hệ thống tự động nhận diện và phiên dịch ngôn ngữ ký hiệu.



Hình 1: Minh họa bài toán nhận diện ngôn ngữ ký hiệu tiếng Việt.

Trong bối cảnh đó, cuộc thi đặt ra cho thí sinh bài toán xây dựng một hệ thống Nhận diện Ngôn ngữ Ký hiệu (Sign Language Recognition - SLR), tận dụng các tiến bộ trên. Cụ thể, nhiệm vụ là phát triển mô hình có thể nhận đầu vào là các video ký hiệu và phân loại chúng thành những nhân từ hoặc cụm từ tiếng Việt tương ứng.

Mục lục

I.	Bối cảnh bài toán	1
II.	Yêu cầu và thể lệ cuộc thi	3
III.	Phân tích dữ liệu	5
III.1.	Phân bố các tập dữ liệu <code>train</code> và <code>test</code>	5
III.2.	Tổng quan về dữ liệu	6
III.3.	Phân bố nhãn và mất cân bằng lớp	9
IV.	Phân tích baseline	11
IV.1.	Xử lý dữ liệu đầu vào	11
IV.2.	Kiến trúc mô hình CRNN	12
IV.3.	Chiến lược huấn luyện	14
V.	Ứng dụng các cải tiến cho dữ liệu	14
V.1.	Chuẩn hoá chuỗi video	15
V.2.	Augmentation video nhất quán	16
V.3.	Lấy mẫu cân bằng cho tập huấn luyện	19
VI.	Ứng dụng các cải tiến cho mô hình	21
VI.1.	Backbone: ConvNeXt-Tiny	22
VI.2.	Mô hình hóa thời gian: Transformer Encoder	22
VI.3.	Tổng hợp thông tin: Attention Pooling	23
VII.	Cải tiến chiến lược huấn luyện	24
VIII.	Các phương pháp mở rộng thêm	26
VIII.1.	Phân tích thêm dữ liệu	26
VIII.2.	Đổi backbone sang Vision Transformer (ViT)	28
VIII.3.	Thay đổi hàm Loss	29
Phụ lục		33

II. Yêu cầu và thể lệ cuộc thi

Các tài nguyên cho bài toán được cung cấp tại [Google Drive](#).

Mô tả bài toán

Cho một tập dữ liệu gồm các video biểu diễn các ký hiệu tay tương ứng với các từ hoặc cụm từ trong ngôn ngữ ký hiệu Việt Nam. Nhiệm vụ của bạn là xây dựng một mô hình phân loại hình ảnh để nhận diện ký hiệu tương ứng.

- **Đầu vào:** Mô hình nhận đầu vào là $X = \{I_1, I_2, \dots, I_n\}$, với mỗi I_i là một hình ảnh tay đang biểu diễn ký hiệu.
- **Mục tiêu:** Dự đoán nhãn y_i tương ứng với từ hoặc cụm từ mà tay đó biểu diễn:

$$\hat{y}_i = f_{\theta}(I_i),$$

trong đó f_{θ} là mô hình học sâu được huấn luyện.

Dữ liệu và mô hình baseline

- **Dữ liệu huấn luyện:** Bộ video ngắn chứa các ký hiệu tay, được gán nhãn tương ứng (ví dụ: các từ như “An ủi”, “Xin lỗi”, “Cảm ơn”, ...).
- **Dữ liệu kiểm thử:** Tập video chưa biết nhãn, dùng để đánh giá mô hình.
- **Mô hình cơ sở:** Thí sinh được cung cấp sẵn một mô hình cơ sở đã được huấn luyện sẵn trên ImageNet CRNN – một kiến trúc mạng nơ-ron được thiết kế cho nhận dạng hành động trong video.
- **Mô hình khác:** Thí sinh cũng có thể sử dụng các mô hình *pretrained* được huấn luyện trên ImageNet.
- **Tính đa dạng:** Các dữ liệu được chụp trong nhiều điều kiện ánh sáng, góc quay, những người thực hiện khác nhau và màu da khác nhau để đảm bảo tính đa dạng.

Yêu cầu

- Huấn luyện mô hình phân loại hình ảnh/chuỗi hình ảnh nhận diện chính xác ký hiệu tay.
- Kết quả đầu ra là nhãn của từ hoặc cụm từ tương ứng với video đầu vào.
- Thí sinh được khuyến khích đề xuất thêm bước xử lý đầu vào (*preprocessing*) hoặc cải tiến mô hình để tăng độ chính xác.

Tiêu chí đánh giá

Tổng thể (Macro-F1): Là giá trị trung bình của chỉ số F_1 trên tất cả các lớp, giúp đánh giá cân bằng giữa Precision và Recall.

Với mỗi lớp (i), các chỉ số được tính như sau:

$$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i}.$$

$$\text{Recall}_i = \frac{TP_i}{TP_i + FN_i}.$$

$$F_{1,i} = 2 \times \frac{\text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}.$$

Trong đó:

- **TP (True Positive):** số mẫu thuộc lớp dương (positive) được dự đoán đúng là positive.
- **TN (True Negative):** số mẫu thuộc lớp âm (negative) được dự đoán đúng là negative.
- **FP (False Positive):** số mẫu thuộc lớp âm nhưng bị mô hình dự đoán nhầm là positive.
- **FN (False Negative):** số mẫu thuộc lớp dương nhưng bị mô hình dự đoán nhầm là negative.

Chỉ số Macro-F1 được định nghĩa:

$$\text{Macro-}F_1 = \frac{1}{N} \sum_{i=1}^N F_{1,i},$$

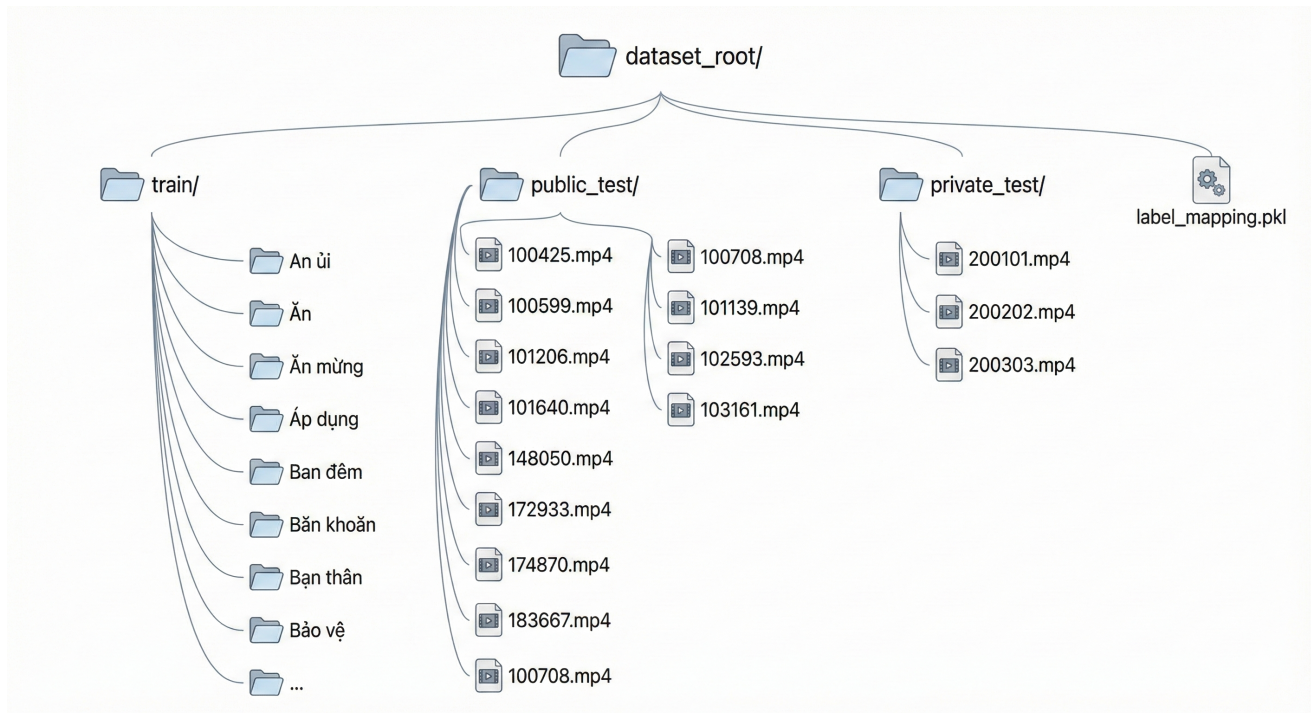
trong đó N là số lớp.

III. Phân tích dữ liệu

Trước khi thiết kế mô hình, bước quan trọng đầu tiên là hiểu rõ dữ liệu mà mô hình sẽ học: loại video này là gì, độ dài ra sao, được gán nhãn như thế nào và phân bố lớp có cân bằng hay không. Phần này trình bày một số phân tích khám phá dữ liệu (EDA) ở cả khía cạnh định tính và định lượng, làm nền tảng cho các quyết định thiết kế mô hình và chiến lược xử lý dữ liệu ở các phần sau.

III.1. Phân bố các tập dữ liệu train và test

Trong bộ dữ liệu mà ban tổ chức cung cấp, thư mục gốc bao gồm ba thư mục chứa video hành động ngôn ngữ ký hiệu và một tệp ánh xạ nhãn, được minh họa ở Hình 2:



Hình 2: Cấu trúc tổ chức bộ dữ liệu được cung cấp bởi ban tổ chức.

- **train/**: chứa toàn bộ dữ liệu có gán nhãn, dùng để huấn luyện và xây dựng mô hình.
- **public_test/**: chứa dữ liệu không có nhãn, dùng để sinh dự đoán gửi lên hệ thống đánh giá công khai.
- **private_test/**: chứa dữ liệu không có nhãn dùng cho đánh giá ẩn (xếp hạng cuối cùng).
- **label_mapping.pkl**: tệp pickle lưu ánh xạ giữa tên lớp (chuỗi tiếng Việt) và chỉ số nguyên tương ứng được mô hình sử dụng trong quá trình huấn luyện và suy luận.

Trong thư mục **train/**, dữ liệu được tổ chức theo dạng mỗi lớp một thư mục con như minh họa ở Hình 2. Mỗi thư mục con mang tên một từ hoặc cụm từ trong ngôn ngữ ký hiệu tiếng Việt (ví

dụ: “Ăn”, “Ăn mừng”, “An ủi”, ...) và chứa các video thể hiện ký hiệu tương ứng. Tổng cộng có **100 lớp** khác nhau trong tập **train**.

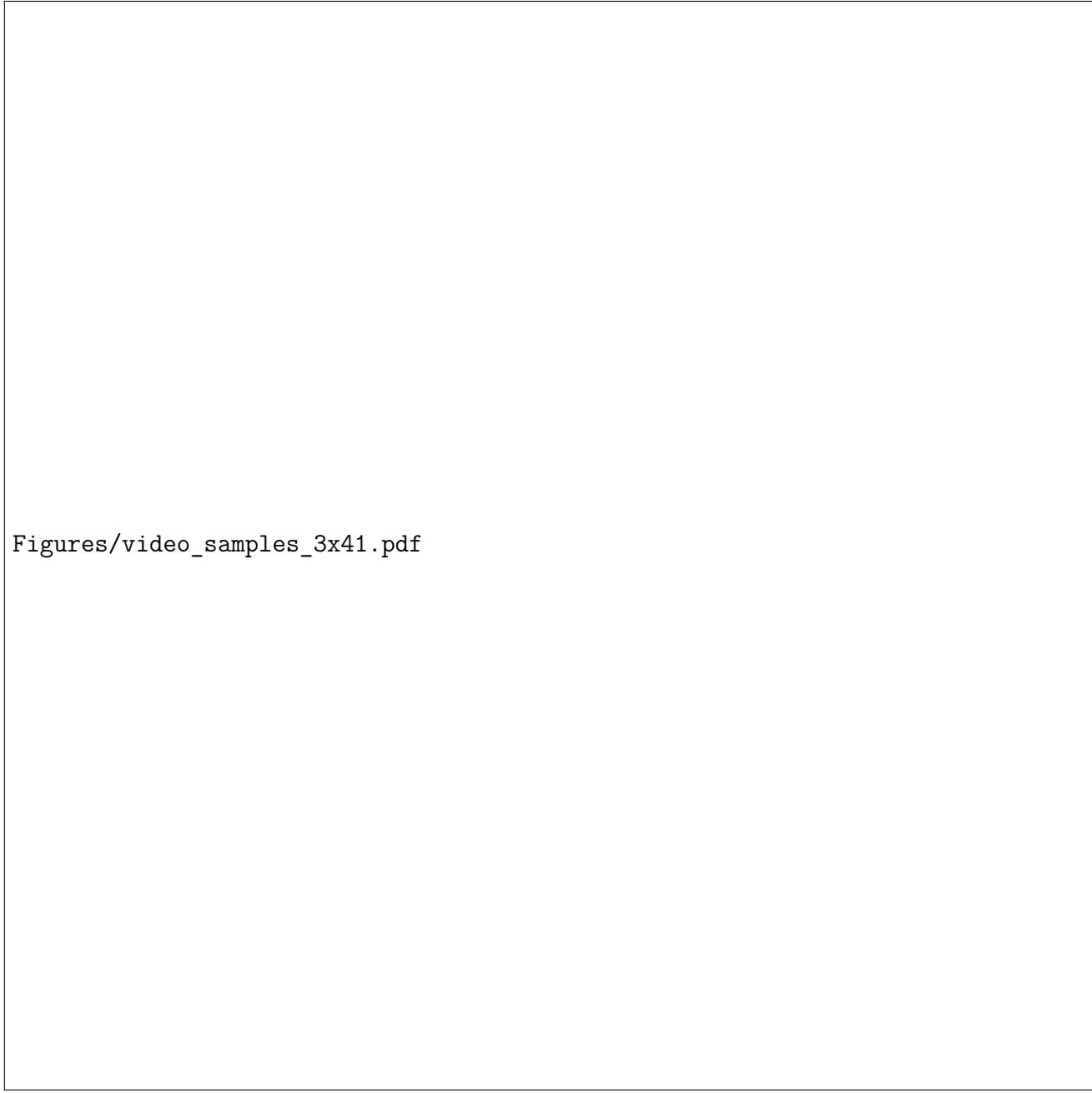
Về quy mô từng tập dữ liệu:

- Tập **train** gồm 3875 video đã được gán nhãn, phân bố vào 100 thư mục lớp như mô tả ở trên.
- Tập **public_test** gồm **1630** video không có nhãn đi kèm.
- Tập **private_test** gồm **2859** video không có nhãn đi kèm.

Như vậy, toàn bộ pipeline xử lý dữ liệu sẽ sử dụng **label_mapping.pkl** để chuyển đổi tên thư mục (tên lớp) thành chỉ số nhãn số nguyên, đọc video từ các thư mục con trong **train/** để huấn luyện và đánh giá mô hình, đồng thời áp dụng cùng quy trình tiền xử lý cho các video trong **public_test/** và **private_test/** khi sinh dự đoán.

III.2. Tổng quan về dữ liệu

Tập dữ liệu bao gồm các video RGB ngắn, mỗi video tương ứng với một từ hoặc cụm từ trong ngôn ngữ ký hiệu tiếng Việt. Để có cái nhìn trực quan, ta trích xuất một số frame đại diện từ nhiều lớp khác nhau như minh hoạ trong Hình 3.



Figures/video_samples_3x41.pdf

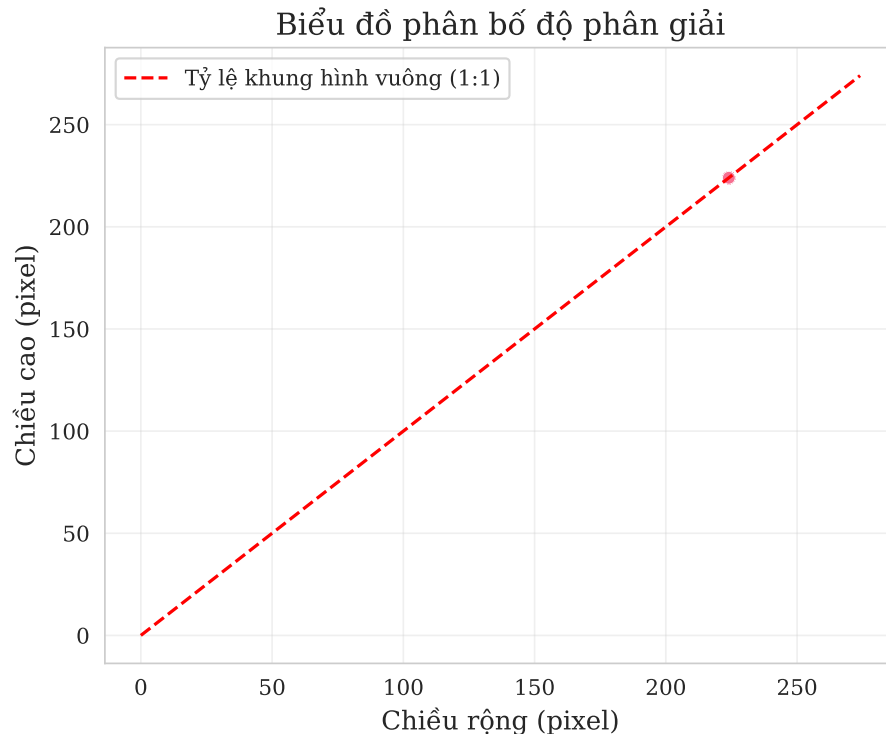
Hình 3: Một số frame trích xuất từ các video thuộc nhiều lớp khác nhau trong tập huấn luyện.
(Hình này được loại bỏ vì chứa thông tin cá nhân; bạn đọc xem dataset để hiểu hơn).

Từ các mẫu frame trực quan, ta rút ra một số đặc điểm chung của dữ liệu:

- **Góc quay và bố cục:** Hầu hết video được quay ở góc chính diện, khung hình bán thân (upper body) giúp tập trung vào chuyển động tay và biểu cảm khuôn mặt - hai yếu tố cốt lõi của ngôn ngữ ký hiệu.
- **Tính đa dạng:** Cùng một ký hiệu được thực hiện bởi nhiều người với giới tính, vóc dáng, màu da, trang phục khác nhau. Điều này giúp mô hình học được đặc trưng bất biến thay vì học theo một cá nhân cụ thể.
- **Bối cảnh và ánh sáng:** Đa số video quay trong nhà với phong nền khá đơn giản, song điều kiện ánh sáng vẫn có độ biến thiên nhất định (video sáng rõ, video hơi tối hoặc ám màu). Mô hình do đó cần có khả năng học được sự thay đổi về ánh sáng và màu sắc.

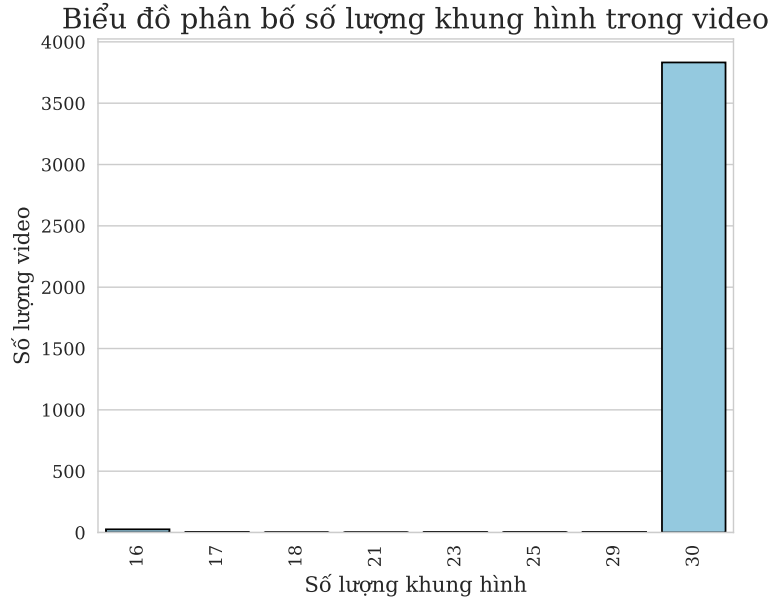
Đặc điểm không gian và thời gian của video

Khác với bài toán ảnh tĩnh, dữ liệu video mang thông tin ở cả hai chiều: không gian (kích thước khung hình) và thời gian (số lượng frame, fps). Việc chuẩn hoá hai yếu tố này là bước quan trọng trước khi đưa vào mô hình.



Hình 4: Biểu đồ phân bố độ phân giải của các video trong tập train.

Hình 4 cho thấy độ phân giải của các video toàn bộ tập trung quanh tại một giá trị kích thước chuẩn 224×224 , phù hợp với model có backbone như ConvNeXt, ... được pretrained trên ImageNet ít gây méo hình và đơn giản hoá bước tiền xử lý.



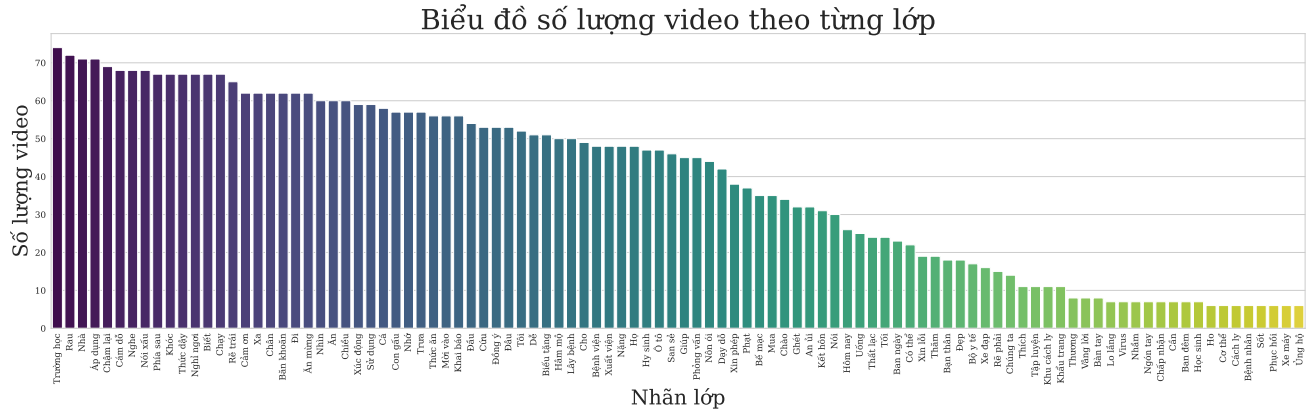
Hình 5: Biểu đồ phân bố số lượng khung hình trong các video huấn luyện.

Quan sát Hình 5, có thể thấy phân bố số lượng khung hình giữa các video gần như “dồn cục” vào một giá trị duy nhất. Cột ở vị trí khoảng 30 frames chiếm áp đảo, trong khi các giá trị còn lại (từ 16-29 frames) chỉ xuất hiện lác đác với tần suất rất nhỏ, gần như khó nhìn thấy trên biểu đồ. Điều này cho thấy chiều dài video trong tập huấn luyện được kiểm soát khá chặt chẽ: phần lớn các mẫu đều có thời lượng tương đương nhau, chỉ tồn tại một số ít ngoại lệ ngắn hơn.

Nói cách khác, về mặt thời gian, bộ dữ liệu huấn luyện có độ biến thiên rất thấp: không có các video quá dài hoặc quá ngắn, và số khung hình của đa số mẫu nằm tập trung quanh một mốc chuẩn duy nhất. Đây là một đặc điểm thuận lợi khi xây dựng pipeline xử lý vì ta không phải đối mặt với sự phân tán mạnh về độ dài chuỗi.

III.3. Phân bố nhãn và mất cân bằng lớp

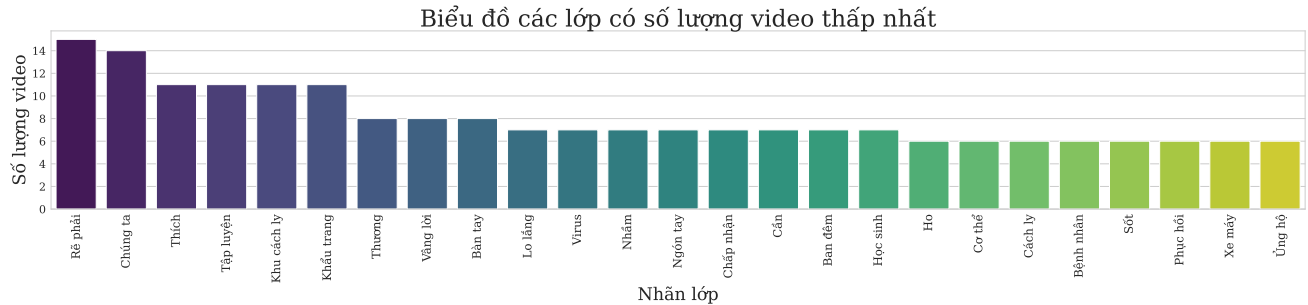
Sau khi hiểu cấu trúc và chất lượng của video, bước tiếp theo là xem cách các video được phân bố theo nhãn. Đây là yếu tố ảnh hưởng trực tiếp tới lựa chọn hàm mất mát và chiến lược lấy mẫu trong quá trình huấn luyện.



Hình 6: Thống kê tổng số video theo từng lớp hành động trong tập huấn luyện.

Hình 6 cho thấy số lượng video giữa các lớp chênh lệch rất lớn. Một vài lớp ở bên trái biểu đồ có tới gần 100 video, trong khi nhiều lớp ở phía bên phải chỉ có vài chục, thậm chí dưới 10 video. Đường cong giảm dần nhanh và kéo dài cho tới cuối trục hoành, thể hiện rõ dạng phân phối “đuôi dài”.

Để nhìn rõ hơn phần đuôi này, ta phóng to các lớp có ít mẫu nhất như trong Hình 7.

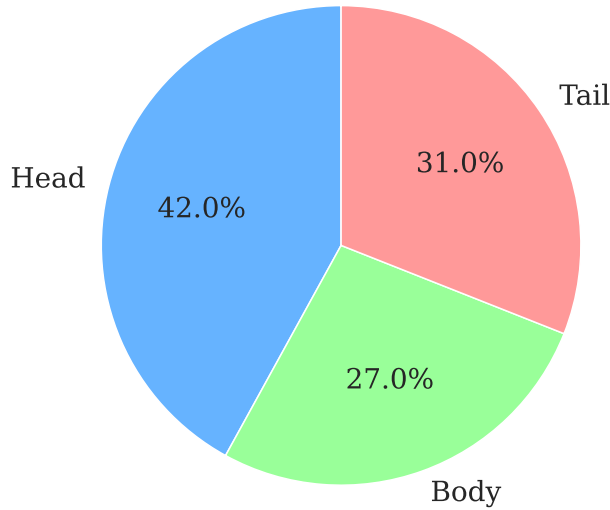


Hình 7: Các lớp có số lượng video thấp nhất trong tập huấn luyện.

Có thể thấy không ít lớp chỉ sở hữu khoảng 4-8 video. Với số lượng mẫu ít như vậy, nếu huấn luyện theo cách thông thường, mô hình rất dễ “bỏ quên” những lớp này và tập trung vào các lớp xuất hiện nhiều hơn.

Để định lượng mức độ mất cân bằng, ta chia các lớp thành ba nhóm dựa trên số lượng video mỗi lớp: nhóm Head gồm các lớp có từ 50 video trở lên, nhóm Body gồm các lớp có từ 20-49 video/lớp, và nhóm Tail gồm các lớp hiếm với dưới 20 video/lớp. Sau đó, ta so tỉ lệ từng nhóm như Hình 8.

Biểu đồ tỷ lệ mất cân bằng lớp



Hình 8: Tỷ lệ giữa nhóm lớp Head (nhiều mẫu), Body và Tail (ít mẫu).

Kết quả cho thấy nhóm Head chỉ chiếm 42% tổng số lớp nhưng nắm phần lớn số lượng video, nhóm Body chiếm 27%, còn nhóm Tail chiếm tới 31% số lớp nhưng mỗi lớp lại có rất ít mẫu. Nói cách khác, gần một phần ba số nhãn trong bài toán rơi vào vùng “hiếm dữ liệu”.

Đây là một thách thức quan trọng đối với việc tối ưu mô hình, đặc biệt khi thước đo mục tiêu là Macro-F1 - vốn coi trọng đều tất cả các lớp. Nếu không có biện pháp xử lý phù hợp, mô hình sẽ thiên lệch về các lớp Head, đạt độ chính xác cao trên những lớp phổ biến nhưng lại dự đoán rất kém đối với những lớp Tail ít dữ liệu.

IV. Phân tích baseline

Sau khi đã hiểu tương đối rõ đặc điểm của dữ liệu qua phần phân tích dữ liệu, bước tiếp theo là phân tích mô hình baseline được cung cấp trong đề bài trước khi đề xuất các hướng cải thiện. Hiểu được baseline một cách vững chắc là bước đầu tiên để đánh giá hiệu quả của các phương pháp cải tiến sau này.

Baseline hiện tại sử dụng kiến trúc CRNN (Convolutional Recurrent Neural Network), kết hợp sức mạnh trích xuất đặc trưng hình ảnh của CNN và khả năng mô hình hóa chuỗi thời gian của RNN (cụ thể hơn là mạng LSTM).

IV.1. Xử lý dữ liệu đầu vào

Dữ liệu video như được nói ở phần EDA trên có đặc thù là có số lượng frame không cố định (từ 16 - 30 frames). Để đưa vào mô hình deep learning, ta cần chuẩn hóa chúng về một định dạng thống nhất.

Quy trình xử lý dữ liệu trong baseline được thực hiện qua các bước sau:

1. Đọc video: Sử dụng thư viện OpenCV để đọc từng frame hình ảnh từ file video.
2. Lấy mẫu: Vì video có độ dài khác nhau, ta cần cố định số lượng frame đầu vào (trong baseline là $T = 16$).
 - Nếu video dài hơn 16 frame: Lấy mẫu đều (uniform sampling) để giữ lại thông tin toàn cục.
 - Nếu video ngắn hơn 16 frame: Lặp lại frame cuối cùng (padding) cho đủ độ dài.
3. Chuẩn hóa: Các frame được đưa về phân phối chuẩn của ImageNet (mean và std) để tận dụng tốt các trọng số pretrained. Các bạn có thể coi cách các thư viện và paper mẫu chuẩn hoá tập dataset ImageNet ở [đây](#).

Dưới đây là đoạn code minh họa việc lấy mẫu frame trong lớp “VideoDataset”:

Hàm lấy mẫu frame

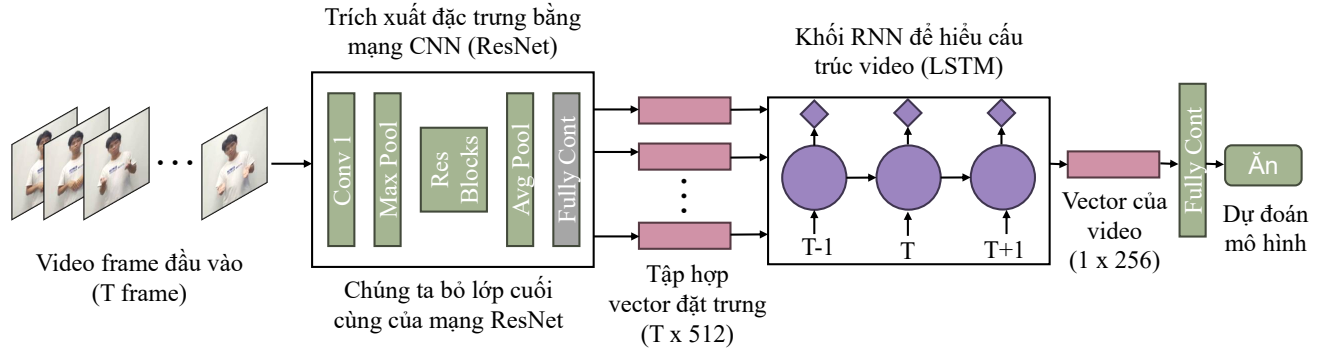
```

1 def _downsample_frames(self, frames):
2     num_frames = frames.shape[0]
3     if num_frames == self.target_frames:
4         return frames
5     elif num_frames < self.target_frames:
6         # Padding nếu thiếu frame
7         pad = self.target_frames - num_frames
8         return torch.cat([frames, frames[-1:].repeat(pad, 1, 1, 1)], dim=0)
9     else:
10        # Lấy mẫu đều nếu thừa frame
11        idx = torch.linspace(0, num_frames - 1, self.target_frames).long()
12        return frames[idx]
```

Tuy nhiên, hàm `_downsample_frames` hiện tại đang xử lý các video ngắn (số frame < 16) bằng cách lặp lại frame cuối cùng. Việc này tạo ra một chuỗi hình ảnh “tĩnh” (đóng băng) ở cuối video. Mạng LSTM có thể hiểu nhầm rằng đặc trưng của hành động bao gồm cả việc “đứng yên” này, gây nhiễu khi phân loại các hành động có tính chuyển động liên tục.

IV.2. Kiến trúc mô hình CRNN

Một trong các mô hình đơn giản nhất để giải quyết bài toán này là mô hình CRNN (Convolutional Recurrent Neural Network). Mô hình được thiết kế theo dạng encoder-decoder với cách hiểu thời gian thông qua cơ chế của mạng LSTM.



Hình 9: Sơ đồ kiến trúc CRNN: ResNet18 trích xuất đặc trưng từng frame, sau đó LSTM tổng hợp thông tin theo thời gian.

Cấu trúc chi tiết bao gồm:

1. Spatial Encoder (CNN): Sử dụng “ResNet18” đã được huấn luyện trước trên ImageNet 1K.
 - Ta loại bỏ lớp Fully Connected cuối cùng để lấy vector đặc trưng.
 - Đầu ra của mỗi frame là một vector có kích thước 512.
2. Temporal Encoder (RNN): Sử dụng mạng “LSTM”.
 - Đầu vào: Chuỗi các vector đặc trưng từ CNN (kích thước $Batch \times 16 \times 512$).
 - LSTM giúp mô hình học được sự thay đổi của hành động theo thời gian (ví dụ: giơ tay lên rồi hạ tay xuống).
 - Hidden size được thiết lập là 256.
3. Classifier: Một lớp tuyến tính (Linear layer) đơn giản để ánh xạ từ trạng thái ẩn cuối cùng của LSTM sang 100 lớp hành động cần phân loại.

Định nghĩa mô hình CRNN

```

1 class CRNN(nn.Module):
2     def __init__(self, num_classes=100, hidden_size=256):
3         super(CRNN, self).__init__()
4         # Sử dụng ResNet18 làm backbone
5         resnet = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
6         self.cnn = nn.Sequential(*list(resnet.children())[:-2])
7
8         # LSTM để xử lý chuỗi thời gian
9         self.rnn = nn.LSTM(512, hidden_size, batch_first=True, dropout=0.3)
10        self.fc = nn.Linear(hidden_size, num_classes)
11
12    def forward(self, x):
13        B, T, C, H, W = x.size()
14        # Gộp Batch và Time để đưa qua CNN
15        x = x.view(B * T, C, H, W)
16        features = self.cnn(x)
17

```

```

18     # Reshape lại để đưa vào RNN
19     seq = features.view(B, T, 512)
20     rnn_out, _ = self.rnn(seq)
21
22     # Lấy output tại bước thời gian cuối cùng
23     return self.fc(rnn_out[:, -1, :])

```

IV.3. Chiến lược huấn luyện

Hàm huấn luyện model

```

1 def train_model(model, train_loader, val_loader,
2                 num_epochs=10, lr=5e-4, device='cuda', save_path='best_model.pth'):
3     model = model.to(device)
4     criterion = nn.CrossEntropyLoss()
5     optimizer = AdamW(model.parameters(), lr=lr, weight_decay=1e-4)
6     scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, 'min',
7                                                             factor=0.5, patience=3)
8     return model

```

Quá trình huấn luyện sử dụng hàm mất mát “CrossEntropyLoss” tiêu chuẩn cho bài toán phân loại đa lớp. Một số điểm đáng chú ý trong cấu hình huấn luyện như sau:

- **Optimizer:** Sử dụng “AdamW” với learning rate khởi tạo là $1e-4$ và weight decay $1e-4$ để tránh overfitting.
- **Scheduler:** Sử dụng “ReduceLROnPlateau”. Nếu loss trên tập validation không giảm sau 3 epoch, learning rate sẽ giảm đi một nửa. Điều này giúp mô hình hội tụ tốt hơn ở các epoch sau trong việc huấn luyện mô hình.
- **Metric:** Đánh giá dựa trên F1-score (Macro average) để đảm bảo sự cân bằng giữa độ chính xác (Precision) và độ phủ (Recall) trên tất cả các lớp hành động.

Hàm **CrossEntropyLoss** mặc định đối xử bình đẳng với tất cả các lớp. Trong bối cảnh F1-score (Macro average) là trọng số đánh giá, việc dự đoán sai một lớp hiếm (minority class) sẽ bị phạt nặng tương đương với lớp phổ biến. Mô hình hiện tại có xu hướng ưu tiên các lớp đa số để tối ưu hóa Accuracy tổng thể, làm giảm F1-score.

V. Ứng dụng các cải tiến cho dữ liệu

Từ các kết quả EDA ở phần phân tích dữ liệu ta có thể nhận thấy dữ liệu video ngôn ngữ ký hiệu có ba đặc điểm nổi bật như sau:

- **Chiều thời gian:** Số khung hình của đa số video gần như tập trung tại 30 frames, chỉ có một số ít ngoại lệ ngắn hơn. Nếu đưa toàn bộ 30 frames vào mô hình vừa dư thừa thông tin, vừa tốn tài nguyên, trong khi các video ngắn lại không đủ độ dài.

- **Chiều không gian:** Video có sự biến thiên về vị trí người biểu diễn ngôn ngữ ký hiệu trong khung hình, điều kiện ánh sáng, màu sắc, tốc độ biểu diễn,... như đã thấy trong các ví dụ Hình 3. Nếu không xử lý tốt, mô hình dễ overfit vào những điều kiện quay cụ thể.
- **Phân bố nhãn:** Biểu đồ phân bố nhãn cho thấy dữ liệu mang dạng phân phối đuôi dài, với nhóm Head có rất nhiều video, trong khi nhóm Tail chỉ có vài mẫu mỗi lớp. Điều này gây thiên lệch khi huấn luyện, đặc biệt khi tiêu chí chấm điểm là Macro-F1 (coi trọng đều các lớp).

Những đặc điểm này dẫn tới ba vấn đề: chuỗi thời gian không đồng nhất, mô hình dễ overfit vào điều kiện quay cụ thể, và lớp hiếm bị “bỏ quên” khi huấn luyện. Dựa trên đó, ta thiết kế lại data pipeline với ba nhóm cải tiến: (i) chuẩn hoá chuỗi video, (ii) tăng cường dữ liệu nhất quán theo thời gian, và (iii) lấy mẫu cân bằng cho tập huấn luyện.

V.1. Chuẩn hoá chuỗi video

Từ phân tích chiều thời gian và độ phân giải, ta quyết định: (1) cố định số frame đầu vào là $T = 16$ để vừa bao phủ toàn bộ diễn biến cử chỉ, vừa giảm dư thừa so với 30 frames gốc; (2) đưa mọi khung hình về kích thước chuẩn 224×224 và chuẩn hoá theo mean/std của ImageNet để tương thích với backbone pretrained trên ImageNet.

Lớp `VideoDataset` chịu trách nhiệm: (i) duyệt cấu trúc thư mục `train/`, (ii) ánh xạ tên lớp sang chỉ số bằng `label_mapping.pkl`, (iii) đọc video, áp dụng augmentation (nếu train), (iv) downsample và chuẩn hoá.

Khởi tạo lớp `VideoDataset`

```

1 class VideoDataset(Dataset):
2
3     # ... Xem thêm tại phần Phụ lục
4
5     def _downsample_frames(self, frames):
6         total = frames.shape[0]
7         if total >= self.target_frames:
8             indices = torch.linspace(0, total - 1, self.target_frames).long()
9         else:
10            indices = torch.arange(total)
11            pad = self.target_frames - total
12            indices = torch.cat([indices, indices[-1].repeat(pad)])
13
14        frames = frames[indices]
15
16        if frames.shape[1] != 224 or frames.shape[2] != 224:
17            frames = frames.permute(0, 3, 1, 2).float()
18            frames = F.interpolate(frames, size=(224, 224),
19                                  mode='bilinear', align_corners=False)
20            frames = frames.permute(0, 2, 3, 1).to(torch.uint8)
21
22        return frames
23
24     def _normalize(self, frames):

```

```

25     """Normalize về ImageNet mean/std"""
26     frames = frames.float() / 255.0
27     frames = frames.permute(0, 3, 1, 2) # (T, H, W, C) -> (T, C, H, W)
28
29     mean = torch.tensor(self.mean).view(1, 3, 1, 1)
30     std = torch.tensor(self.std).view(1, 3, 1, 1)
31     frames = (frames - mean) / std
32
33     return frames

```

Ở đây, hàm `_downsample_frames` tạo ra các chỉ số cách đều trên trục thời gian, giúp “nén” toàn bộ diễn biến hành động thành đúng $T = 16$ frame đại diện nhưng vẫn giữ được thông tin từ đầu đến cuối video (uniform sampling).

```

1 indices = torch.linspace(0, total - 1, self.target\_frames).long()

```

Hàm `_normalize` nhằm chuẩn hóa phân theo phân phối của ImageNet, `_normalize` trước tiên đưa giá trị pixel về khoảng $[0, 1]$, sau đó chuyển thứ tự chiều từ (T, H, W, C) sang (T, C, H, W) để phù hợp chuẩn PyTorch. Tiếp theo, từng kênh màu được chuẩn hoá bằng bộ `mean/std` của ImageNet:

$$\text{frames} \leftarrow \frac{\text{frames} - \text{mean}}{\text{std}}.$$

Nhờ vậy, phân phối pixel của dữ liệu ngôn ngữ ký hiệu được đưa về cùng không gian thống kê với tập ImageNet, giúp tận dụng hiệu quả hơn các trọng số pretrained của backbone pretrained trên ImageNet khi trích xuất đặc trưng từ từng frame.

V.2. Augmentation video nhất quán

Từ phần tổng quan dữ liệu (Hình 3), ta thấy các video khác nhau đáng kể về tốc độ biểu diễn ký hiệu, vị trí người biểu diễn trong khung hình và điều kiện ánh sáng. Để mô hình học được các đặc trưng bền vững trước những thay đổi này, ta cần tăng cường dữ liệu. Tuy nhiên, nếu áp dụng augmentation độc lập trên từng frame, chuyển động trong video sẽ bị méo (rung, giật), khiến mô hình khó nắm bắt các đặc trưng.

Vì vậy, ta xây dựng lớp `VideoAugmentation` với nguyên tắc: các tham số biến đổi được sinh một lần và áp dụng đồng nhất cho toàn bộ chuỗi frame, bảo toàn tính liên tục theo thời gian.

Khởi tạo lớp `VideoAugmentation` với các phương pháp Speed, Crop, Color jitter

```

1 class VideoAugmentation:
2     def __init__(self,
3                 crop_scale=(0.85, 1.0),
4                 brightness=0.2,
5                 contrast=0.2,
6                 saturation=0.2,
7                 speed_range=(0.9, 1.1)):

```

```

8
9     self.crop_scale = crop_scale
10    self.brightness = brightness
11    self.contrast = contrast
12    self.saturation = saturation
13    self.speed_range = speed_range
14
15    # ... Xem thêm tại phần Phụ lục
16
17    def _speed_augment(self, frames):
18        """Thay đổi tốc độ video bằng cách resample frames"""
19        T = frames.shape[0]
20        speed = random.uniform(self.speed_range[0], self.speed_range[1])
21
22        new_T = int(T / speed)
23        if new_T < 4:
24            new_T = 4
25        if new_T == T:
26            return frames
27
28        # Resample frames
29        indices = torch.linspace(0, T - 1, new_T).long()
30        indices = torch.clamp(indices, 0, T - 1)
31        frames = frames[indices]
32
33        return frames

```

Hàm `_speed_augment` điều chỉnh tốc độ “ảo” của video bằng cách thay đổi số lượng frame hữu hiệu thông qua tham số `speed` được rút ngẫu nhiên trong khoảng `speed_range`. Khi `speed < 1`, số frame đầu ra tăng lên, mô phỏng việc người biểu diễn ký chậm hơn; khi `speed > 1`, số frame giảm xuống, tương ứng một động tác được thực hiện nhanh hơn. Việc nội suy chỉ số frame bằng:

_speed_augment:

```

1 indices = torch.linspace(0, T - 1, new_T)

```

giúp giữ nguyên thứ tự thời gian nhưng nén/giãn trục thời gian một cách mượt mà, phản ánh đúng thực tế rằng cùng một ký hiệu có thể được thực hiện với tốc độ rất khác nhau giữa các video.

Khởi tạo lớp VideoAugmentation với các phương pháp Speed, Crop, Color jitter

```

1 class VideoAugmentation:
2     def __init__(self,
3         crop_scale=(0.85, 1.0),
4         brightness=0.2,
5         contrast=0.2,
6         saturation=0.2,
7         speed_range=(0.9, 1.1)):
8

```

```

9         self.crop_scale = crop_scale
10        self.brightness = brightness
11        self.contrast = contrast
12        self.saturation = saturation
13        self.speed_range = speed_range
14
15        # ... Xem thêm tại phần Phụ lục
16
17        def _random_resized_crop(self, frames):
18            """Random crop rồi resize về 224x224"""
19            T, H, W, C = frames.shape
20
21            # Random scale và position
22            scale = random.uniform(self.crop_scale[0], self.crop_scale[1])
23            crop_h, crop_w = int(H * scale), int(W * scale)
24            top = random.randint(0, H - crop_h)
25            left = random.randint(0, W - crop_w)
26
27            # Crop tất cả frames
28            frames = frames[:, top:top+crop_h, left:left+crop_w, :]
29            # Resize về 224x224
30            frames = frames.permute(0, 3, 1, 2).float()
31            frames = F.interpolate(frames, size=(224, 224),
32                                   mode='bilinear', align_corners=False)
33            # (T, C, H, W) -> (T, H, W, C)
34            frames = frames.permute(0, 2, 3, 1)
35
36            return frames.to(torch.uint8)

```

Trong `_random_resized_crop`, hệ số `scale = random.uniform(crop_scale[0], crop_scale[1])` cùng vị trí `top`, `left` được lấy ngẫu nhiên một lần, sau đó áp dụng cho toàn bộ tensor `frames`. Cách làm này mô phỏng việc người biểu diễn đứng gần/xa hoặc lệch trái/phải so với camera, đồng thời đảm bảo mọi frame đều bị cắt ở cùng một vùng. Nhờ đó, quỹ đạo chuyển động của tay vẫn liên tục và mạch lạc theo thời gian, thay vì “nhảy” vị trí nếu ta crop từng frame một cách độc lập.

Khởi tạo lớp VideoAugmentation với các phương pháp Speed, Crop, Color jitter

```

1 class VideoAugmentation:
2     def __init__(self,
3                 crop_scale=(0.85, 1.0),
4                 brightness=0.2,
5                 contrast=0.2,
6                 saturation=0.2,
7                 speed_range=(0.9, 1.1)):
8
9         self.crop_scale = crop_scale
10        self.brightness = brightness
11        self.contrast = contrast
12        self.saturation = saturation
13        self.speed_range = speed_range

```

```

14
15 # ... Xem thêm tại phần Phụ lục
16 def _color_jitter(self, frames):
17     """Color jitter"""
18     # Random parameters
19     brightness_factor = 1.0 + random.uniform(-self.brightness, self.brightness)
20     contrast_factor = 1.0 + random.uniform(-self.contrast, self.contrast)
21     saturation_factor = 1.0 + random.uniform(-self.saturation, self.saturation)
22
23     frames = frames.float()
24
25     # Brightness
26     frames = frames * brightness_factor
27     # Contrast
28     mean = frames.mean(dim=(1, 2), keepdim=True)
29     frames = (frames - mean) * contrast_factor + mean
30     # Saturation
31     gray = frames.mean(dim=-1, keepdim=True)
32     frames = gray + (frames - gray) * saturation_factor
33     # Clamp to valid range
34     frames = torch.clamp(frames, 0, 255)
35
36     return frames.to(torch.uint8)

```

Hàm `_color_jitter` tạo ra các biến thiên về độ sáng, độ tương phản và độ bão hoà thông qua ba hệ số `brightness_factor`, `contrast_factor`, `saturation_factor` được khởi tạo một lần cho toàn bộ video. Điều này cho phép mô phỏng sự khác biệt về điều kiện chiếu sáng, tông màu da hay độ sáng phòng mà ta quan sát được trong Hình 3, nhưng vẫn tránh được hiện tượng “nhấp nháy” khó chịu nếu mỗi frame bị chỉnh màu khác nhau. Kết quả là mô hình được khuyến khích tập trung vào hình thái cử chỉ thay vì phụ thuộc vào một cấu hình ánh sáng cố định.

Khi ghép vào `VideoDataset`, lớp `VideoAugmentation` chỉ được kích hoạt khi `training=True`. Điều này đảm bảo rằng tập huấn luyện được làm giàu bằng các biến thiên thực tế (tốc độ, vị trí, ánh sáng), trong khi tập validation và test vẫn giữ nguyên phân phối gốc của dữ liệu, giúp việc đánh giá mô hình phản ánh đúng chất lượng tổng quát hoá trên dữ liệu thật.

V.3. Lấy mẫu cân bằng cho tập huấn luyện

Cuối cùng, phân tích ở Hình 6-8 cho thấy gần một phần ba số lớp thuộc nhóm Tail với rất ít video. Nếu giữ nguyên sampling ngẫu nhiên, các lớp này hiếm khi xuất hiện trong mini-batch, trong khi Macro-F1 lại coi trọng đều mọi lớp.

Để khắc phục, ta sử dụng `WeightedRandomSampler` với trọng số tỉ lệ nghịch với tần suất xuất hiện của từng lớp, qua đó tăng xác suất chọn các mẫu thuộc lớp hiếm trong quá trình huấn luyện - nói cách khác, lớp nào có ít mẫu trong tập train thì sẽ được “ưu tiên bốc” thường xuyên hơn khi tạo batch.

Balanced Sampler cho tập train

```

1 def create_balanced_sampler(dataset):
2     if hasattr(dataset, 'dataset'):
3         all_labels = [dataset.dataset.label_idx[i] for i in dataset.indices]
4     else:
5         all_labels = dataset.label_idx
6
7     class_counts = np.bincount(all_labels)
8     class_weights = 1.0 / class_counts
9     sample_weights = [class_weights[label] for label in all_labels]
10    sample_weights = torch.FloatTensor(sample_weights)
11
12    sampler = WeightedRandomSampler(
13        weights=sample_weights,
14        num_samples=len(sample_weights),
15        replacement=True
16    )
17
18    print(f"Balanced Sampler: class counts min={class_counts.min()},
19          max={class_counts.max()}")
20    return sampler

```

Đoạn code trên hiện thực hoá đúng ý tưởng “càng hiếm càng được ưu tiên”:

- `all_labels` là danh sách nhãn (dạng chỉ số) của toàn bộ mẫu trong tập train hiện tại. Nếu `dataset` là một `Subset`, ta truy ngược về `dataset.dataset` và lấy theo `indices` để không làm sai lệch tần suất.
- `class_counts = np.bincount(all_labels)` cho ta số lượng mẫu của từng lớp `c`. Đây chính là thống kê mà ta đã hình dung thông qua các biểu đồ phân bố nhãn ở phần EDA.
- `class_weights = 1.0 / class_counts` chuyển tần suất thành trọng số nghịch đảo: lớp có ít mẫu (`class_counts` nhỏ) sẽ nhận trọng số lớn, lớp có nhiều mẫu thì ngược lại.
- `sample_weights` gán trọng số lớp tương ứng cho từng mẫu cụ thể; `WeightedRandomSampler` sau đó sử dụng các trọng số này để quyết định xác suất một mẫu được chọn vào batch.
- Tham số `replacement=True` cho phép cùng một mẫu được chọn nhiều lần trong một epoch nếu nó thuộc lớp rất hiếm, qua đó “bù” lại số lượng dữ liệu ít ỏi của lớp đó.

Sampler này được gắn trực tiếp vào `DataLoader` dành cho huấn luyện, thay thế cho cơ chế `shuffle=True` ngẫu nhiên thuần túy:

Khởi tạo `DataLoader` với balanced sampler

```

1 # ... Xem thêm tại phần Phụ lục
2
3 train_size = int(0.8 * len(train_dataset_base))
4 val_size = len(train_dataset_base) - train_size
5
6 indices = list(range(len(train_dataset_base)))

```



```

7 np.random.seed(42)
8 np.random.shuffle(indices)
9 train_indices = indices[:train_size]
10 val_indices = indices[train_size:]
11
12 train_dataset = torch.utils.data.Subset(train_dataset_base, train_indices)
13 val_dataset = torch.utils.data.Subset(val_dataset_base, val_indices)
14
15 balanced_sampler = create_balanced_sampler(train_dataset)
16
17 train_loader = DataLoader(
18     train_dataset,
19     batch_size=16,
20     sampler=balanced_sampler,
21     collate_fn=collate_fn,
22     num_workers=4
23 )
24
25 val_loader = DataLoader(
26     val_dataset,
27     batch_size=16,
28     shuffle=False,
29     collate_fn=collate_fn,
30     num_workers=4
31 )

```

Nhờ cơ chế này, các lớp Tail tuy có rất ít video trong toàn bộ tập train nhưng lại xuất hiện với tần suất hợp lý hơn trong từng mini-batch. Điều đó giúp gradient từ các lớp hiếm không bị “chìm” so với các lớp Head/Body, và mô hình buộc phải học ranh giới quyết định cho cả những lớp ít dữ liệu. Cách lấy mẫu cân bằng này vì thế phù hợp với tiêu chí Macro-F1: mỗi lớp - dù phổ biến hay hiếm - đều có cơ hội đóng góp công bằng vào hiệu năng cuối cùng của mô hình.

VI. Ứng dụng các cải tiến cho mô hình

Mặc dù mô hình CRNN (ResNet + LSTM) là một baseline phổ biến và đơn giản, nó tồn tại một số hạn chế:

- **Backbone ResNet18:** Ra đời từ 2015, khả năng trích xuất đặc trưng không mạnh mẽ bằng các kiến trúc hiện đại.
- **LSTM:** Xử lý tuần tự khiến việc huấn luyện chậm và gặp khó khăn trong việc nắm bắt các phụ thuộc xa (long-range dependencies) trong chuỗi video dài. Ngoài ra, việc chỉ lấy trạng thái ẩn cuối cùng (last hidden state) để bỏ sót thông tin nếu hành động quan trọng diễn ra ở giữa video.

Để khắc phục, chúng tôi đề xuất kiến trúc **ConvNeXt-Transformer** với ba cải tiến chính: (1) Backbone ConvNeXt-Tiny mạnh mẽ hơn, (2) Transformer Encoder thay thế LSTM để mô hình hóa thời gian, và (3) Cơ chế Attention Pooling để tổng hợp thông tin.

VI.1. Backbone: ConvNeXt-Tiny

Thay vì ResNet18, chúng ta sẽ sử dụng **ConvNeXt-Tiny** để thay thế. Đây là kiến trúc CNN hiện đại được thiết kế lại dựa trên các nguyên lý của Vision Transformer (như patchify stem, depthwise convolution, layer normalization, v.v.).

💡 Tại sao chúng ta tận dụng backbone?

Một thách thức lớn trong bài toán này là số lượng dữ liệu huấn luyện khá hạn chế so với độ phức tạp của không gian đặc trưng video. Nếu huấn luyện một mạng nơ-ron sâu từ đầu (training from scratch), mô hình sẽ phải học hàng triệu tham số chỉ từ vài nghìn video mẫu, dẫn đến nguy cơ “overfitting” rất cao và khả năng tổng quát hóa kém.

Thay vào đó, chiến lược tối ưu là sử dụng “Transfer Learning” (Tạm dịch: học chuyển giao):

- **Tận dụng tri thức sẵn có:** Backbone ConvNeXt-Tiny đã được huấn luyện trên tập dữ liệu khổng lồ ImageNet (1.2 triệu ảnh), do đó nó đã học được các bộ lọc (filters) để nhận diện cạnh, hình dạng, và kết cấu vật thể một cách xuất sắc.
- **Tối thiểu hóa tham số cần học:** Thay vì học lại cách “nhìn” hình ảnh từ con số 0, mô hình chỉ cần tinh chỉnh (fine-tune) một lượng nhỏ trọng số để thích nghi với miền dữ liệu ngôn ngữ ký hiệu.
- **Hội tụ nhanh hơn:** Việc khởi tạo trọng số từ pre-trained model giúp loss giảm nhanh hơn và ổn định hơn so với khởi tạo ngẫu nhiên.

Do đó, kiến trúc ConvNeXt-Transformer ở đây được thiết kế để “đứng trên vai người khổng lồ”, tập trung năng lực tính toán vào việc học mối quan hệ thời gian thay vì tốn tài nguyên để trích xuất đặc trưng thị giác cơ bản mà các paper tốt nhất hiện có đang làm.

Về mặt hiệu năng thực tế, mặc dù mang tên “Tiny”, ConvNeXt vẫn sở hữu khả năng trích xuất đặc trưng vượt trội, đạt độ chính xác cao hơn cả ResNet50 trên tập ImageNet trong khi vẫn duy trì chi phí tính toán thấp hơn. Quan trọng hơn, đầu ra của backbone này cung cấp vector đặc trưng có kích thước $D = 768$, giàu thông tin ngữ nghĩa hơn rất nhiều so với kích thước 512 của ResNet18, tạo tiền đề vững chắc cho các lớp xử lý thời gian phía sau hoạt động hiệu quả.

VI.2. Mô hình hóa thời gian: Transformer Encoder

Tương tự như cách chúng ta đổi qua mô hình ConvNeXt ở backbone, để mô hình được hiểu cấu trúc video tốt hơn ta cũng cần đổi cách mô hình hóa thời gian bằng **Transformer Encoder**. Cơ chế Self-Attention cho phép mô hình nhìn thấy toàn bộ chuỗi frame cùng lúc, từ đó học được mối quan hệ giữa các frame bất kể khoảng cách thời gian. Vì Transformer không có khái niệm về thứ tự, chúng ta phải bổ sung **Positional Encoding** vào chuỗi đặc trưng trước khi đưa vào encoder. Các làm được miêu tả như code sau:

Positional Encoding và Transformer

```
1 class PositionalEncoding(nn.Module):
2     """Positional encoding cho temporal sequence"""
3     def __init__(self, d_model, max_len=64, dropout=0.1):
```

```

4         super().__init__()
5         # ... (Khởi tạo ma trận sin/cos) ...
6         self.register_buffer('pe', pe)
7
8     def forward(self, x):
9         # x: (B, T, d_model)
10        x = x + self.pe[:, :x.size(1), :]
11        return self.dropout(x)
12
13    # Trong ConvNeXtTransformer:
14    # 2. Positional Encoding
15    self.pos_encoder = PositionalEncoding(d_model=768, max_len=64, dropout=0.1)
16
17    # 3. Transformer Encoder
18    encoder_layer = nn.TransformerEncoderLayer(
19        d_model=768, nhead=8, dim_feedforward=768*4,
20        dropout=0.3, activation='gelu', batch_first=True, norm_first=True
21    )
22    self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=2)

```

VI.3. Tổng hợp thông tin: Attention Pooling

Baseline CRNN sử dụng trạng thái ẩn tại bước thời gian cuối cùng (h_T) để phân loại. Điều này rất rủi ro đối với video ngôn ngữ ký hiệu, vì frame cuối cùng thường là lúc người biểu diễn đã hạ tay xuống (trạng thái nghỉ), không chứa thông tin đặc trưng của từ vựng.

Giải quyết việc này ta sẽ sử dụng cơ chế **Attention Pooling**. Mạng sẽ học một trọng số a_t cho mỗi frame t , thể hiện mức độ quan trọng của frame đó. Vector đại diện cho cả video V là tổng có trọng số của các frame:

$$a_t = \text{softmax}(W_2 \tanh(W_1 x_t))$$

$$V = \sum_{t=1}^T a_t x_t$$

Cơ chế Attention Pooling

```

1 class AttentionPooling(nn.Module):
2     def __init__(self, dim):
3         super().__init__()
4         self.attention = nn.Sequential(
5             nn.Linear(dim, dim // 4),
6             nn.Tanh(),
7             nn.Linear(dim // 4, 1)
8         )
9
10    def forward(self, x):
11        # x: (B, T, dim)
12        attn_weights = self.attention(x) # (B, T, 1)

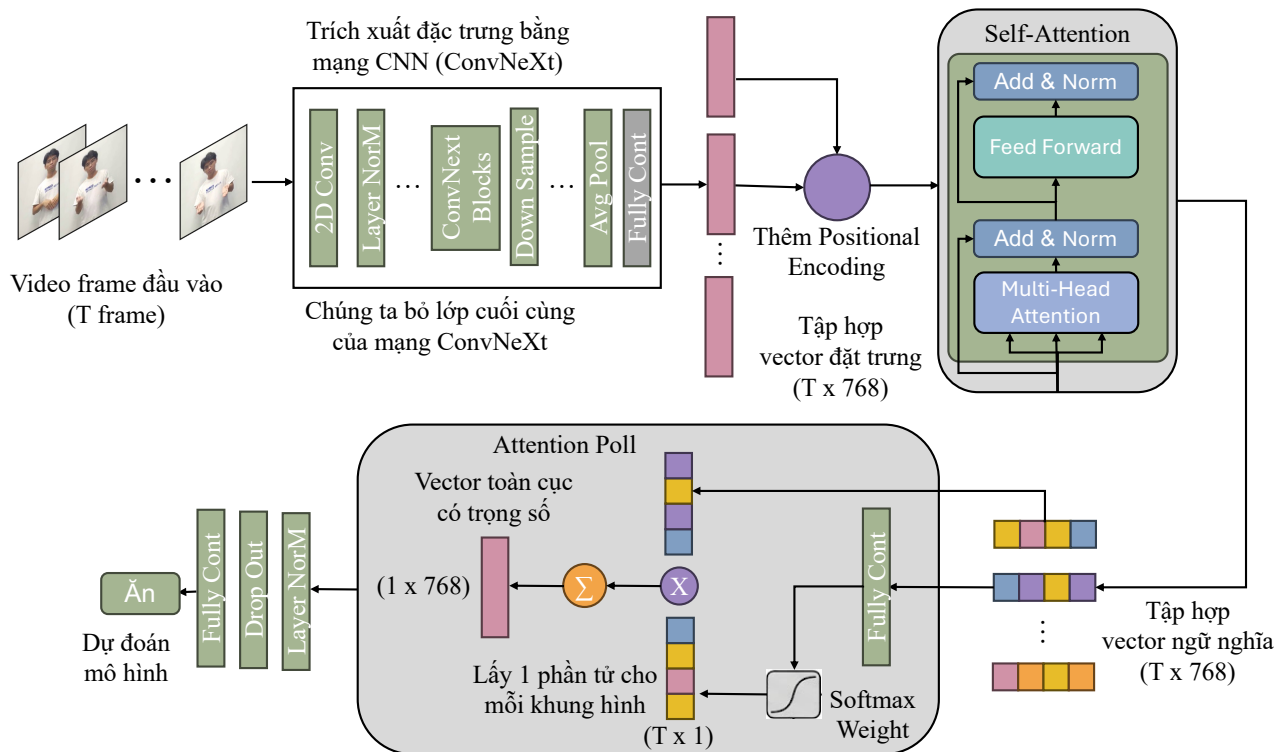
```

```

13     attn_weights = F.softmax(attn_weights, dim=1)
14     pooled = torch.sum(attn_weights * x, dim=1) # (B, dim)
15     return pooled

```

Nhờ cơ chế này, mô hình có thể tự động “tập trung” vào các frame giữa video nơi cử chỉ diễn ra rõ ràng nhất và “lờ đi” các frame đầu/cuối không quan trọng. Hơn nữa, với cấu trúc Transformer, việc sử dụng Attention Pooling sẽ lưu trữ được những thông tin quan trọng nhất cho các layer sau.



Hình 10: Sơ đồ kiến trúc mạng cải tiến: ConvNeXt trích xuất đặc trưng từng frame, sau đó cơ chế Attention giúp các vector đặc trưng nói trên có thêm ngữ nghĩa. Tiếp theo, chúng ta Pooling toàn bộ chuỗi thời gian thành 1 vector duy nhất và dùng FC để thực hiện phân loại.

VII. Cải tiến chiến lược huấn luyện

Bên cạnh kiến trúc, chiến lược huấn luyện cũng được tinh chỉnh để phù hợp với dữ liệu và mô hình mới. Mà ở đây, chúng ta cần lưu ý nhất là hai hiện tượng: dữ liệu mất cân bằng và quá ít (dễ dẫn tới overfit).

Trong bài toán phân loại với nhiều lớp (100 lớp) và dữ liệu có độ nhiễu nhất định (do góc quay, ánh sáng), việc sử dụng one-hot encoding cứng (1 cho đúng, 0 cho sai) dễ dẫn đến overfitting. Mô hình trở nên quá tự tin vào dự đoán của mình. Áp dụng **Label Smoothing** với hệ số $\epsilon = 0.1$ là cách để giải quyết vấn đề này một cách đơn giản nhất. Hàm loss sẽ không ép xác suất lớp đúng phải là 1.0 tuyệt đối, mà chỉ cần đạt $1.0 - \epsilon$. Điều này giúp mô hình tổng quát hóa tốt hơn trên

tập test.

Hàm loss với Label Smoothing

```
1 criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
```

Và cũng để giải quyết vấn đề làm sao để mô hình không bị quá khớp với dữ liệu, chúng ta cũng sẽ thay thế thuật toán tối ưu **Adam** bằng **AdamW**. Adam Weight Decay hoạt động bằng cách thêm một “án phạt” vào hàm mất mát. Thay vì chỉ tối thiểu hóa sai số dự đoán, mô hình buộc phải tối thiểu hóa cả độ lớn của các trọng số. Khi trọng số của mô hình được giữ nhỏ, chúng ta sẽ giảm được độ phức tạp của mô hình qua đó giảm được hiện tượng overfitting. Tham số weight decay được sử dụng là $1e - 4$. Tiếp theo, chiến lược điều chỉnh learning rate **ReduceLROnPlateau** được giữ nguyên.

Cấu hình huấn luyện

```
1 optimizer = AdamW(model.parameters(), lr=1e-4, weight_decay=1e-4)
2 scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
3     optimizer, 'min', factor=0.5, patience=3
4 )
```

VII.0.1. Kết quả xếp hạng

#	Thí sinh	Ngày	ID	Total Score	Public CV Score	Private CV Score
	Không phải TQKhang	2025-11-23 03:56	6590	160.37	83.49	76.88
	Thai Phu An	2025-11-24 00:47	6654	159.76	81.84	77.92
	WrongAnswer	2025-11-23 18:44	6618	157.99	82.96	75.03
	Konichichi	2025-11-20 04:04	6175	146.35	73.16	73.18
	hieupm123	2025-11-21 09:00	6323	143.52	72.12	71.39
6	perry	2025-11-27 05:44	6882	139.26	69.67	69.58
7	Nguyễn Minh Hiếu	2025-12-01 01:49	7227	137.32	69.03	68.29
8	aiteam	2025-11-30 23:07	7211	137.15	69.07	68.08

Hình 11: Kết quả xếp hạng sau khi áp dụng các bước cải tiến trên trong cuộc thi.

Tổng hợp lại, sự kết hợp giữa pipeline dữ liệu (Augmentation + Balanced Sampling) và kiến trúc mô hình (ConvNeXt + Transformer + Attention Pooling) đã đưa về kết quả khả quan trên hệ thống chấm điểm trực tuyến của ban tổ chức Sau khi huấn luyện xong, mô hình được dùng để sinh dự đoán cho hai thư mục video ngôn ngữ ký hiệu public_test và private_test theo định

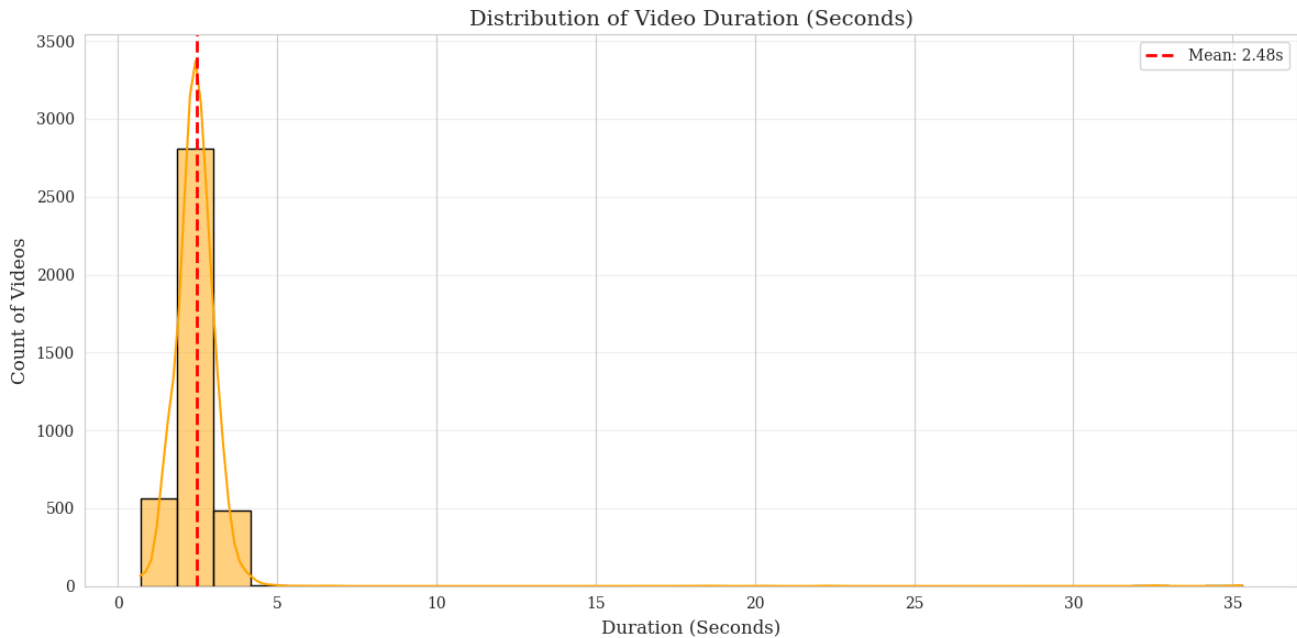
dạng được yêu cầu, đưa lên hệ thống và thu được điểm tổng 137.15, xếp hạng 9 như thể hiện trên bảng xếp hạng. Kết quả này cho thấy thiết kế kiến trúc và quy trình tiền xử lý dữ liệu huấn luyện bị mất cân bằng là phù hợp, đủ sức cạnh tranh với các phương pháp khác trong cuộc thi nhận diện ngôn ngữ ký hiệu.

VIII. Các phương pháp mở rộng thêm

VIII.1. Phân tích thêm dữ liệu

Sau khi đã có một pipeline chạy ổn định và một mô hình ConvNeXt–Transformer cho kết quả khá tốt, một hướng mở rộng tự nhiên là quay lại bước EDA để quan sát dữ liệu kỹ hơn: thời lượng video, các lỗi kỹ thuật trong khung hình và những mẫu trùng lặp. Từ đó có thể đưa ra các quyết định xử lý dữ liệu phù hợp hơn trước khi thử các kiến trúc hoặc thuật toán mới.

Thời lượng video



Hình 12: Phân bố thời lượng video trong tập huấn luyện.

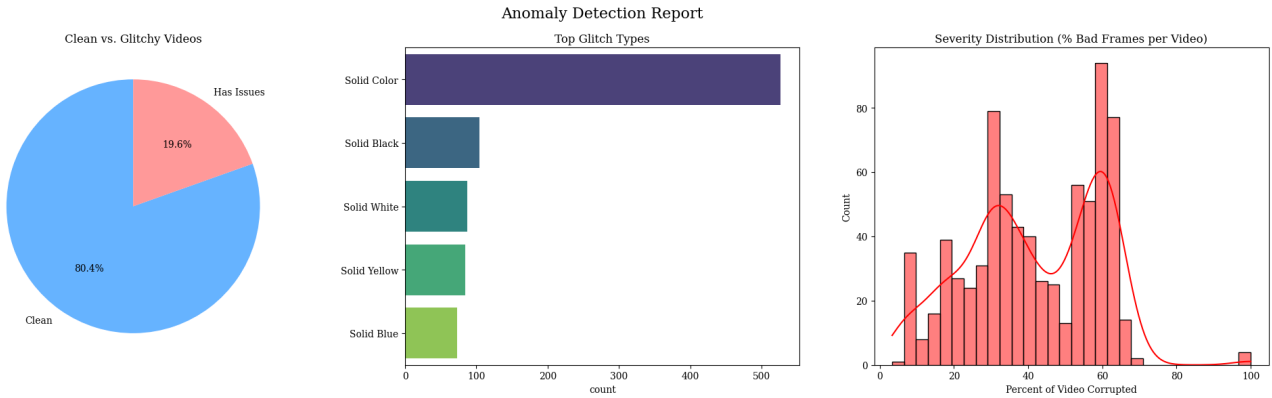
Hình 12 (biểu đồ phân bố thời lượng video) cho thấy phần lớn video chỉ kéo dài quanh mức 2–3 giây, với giá trị trung bình xấp xỉ 2.5 giây. Dải phân bố rất hẹp: gần như mọi video đều nằm trong vùng này, chỉ có một vài điểm lẻ loi kéo dài tới hơn 30 giây.

Từ đó có thể cân nhắc hai lựa chọn:

- Giữ nguyên chiến lược chuẩn hoá $T = 16$ frame, vì đa số video có thời lượng tương đương nên việc nén chúng về cùng một số frame vẫn hợp lý.

- Với những video dài bất thường (hơn 20–30 giây) có thể xem xét cắt chỉ giữ lại đoạn giữa nơi diễn ra hành động, hoặc loại khỏi tập huấn luyện nếu nội dung thực tế khó xác định.

Video lỗi



Hình 13: Phân bố mức độ hỏng theo phần trăm số khung hình lỗi trên mỗi video.

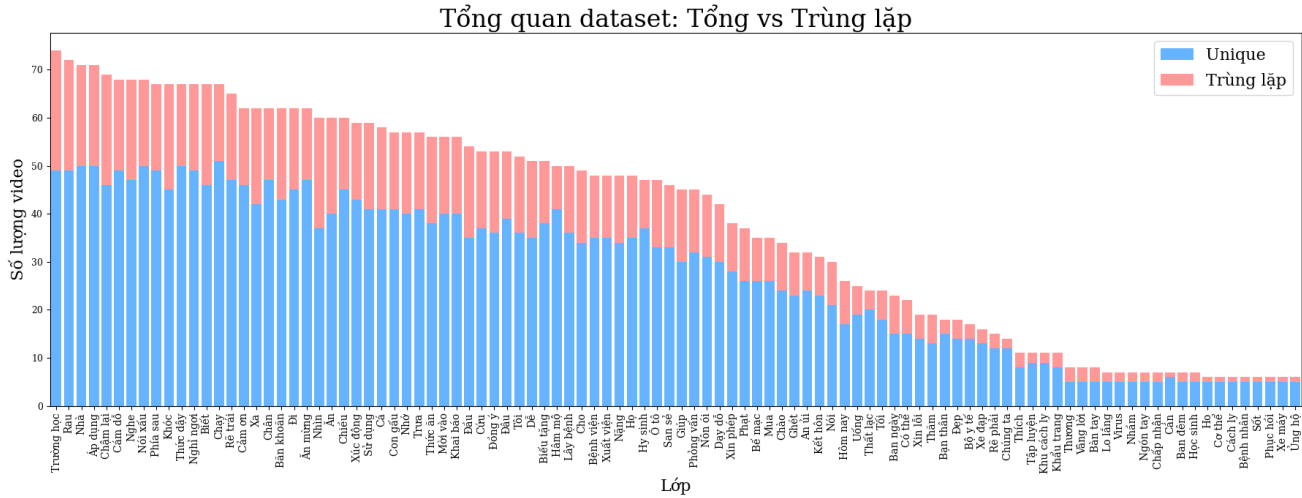
Một hướng EDA khác là quét lại bộ dữ liệu gốc để tìm các khung hình bất thường: toàn màu đen, toàn trắng, chỉ một màu, hoặc bị lỗi mã hoá khiến hình ảnh biến dạng. Kết quả được tóm tắt trong Hình 13.

- Khoảng 80% video sạch, không phát hiện vấn đề rõ rệt.
- Gần 20% video có lỗi, trong đó dạng phổ biến nhất là các khung hình solid color kéo dài (đen, trắng, vàng, xanh).
- Biểu đồ phân bố tỷ lệ khung hình hỏng trên mỗi video cho thấy hai cụm chính: một nhóm quanh mức 30–40%, và một nhóm nặng hơn quanh 60%.

Nếu bỏ qua hoàn toàn các lỗi này, mô hình sẽ học trên những frame không chứa thông tin ngôn ngữ ký hiệu, vô tình coi nhiều là dữ liệu thật. Dựa trên kết quả đó, có thể thử:

- Loại bỏ các frame hỏng trước khi downsample: khi phát hiện frame toàn màu hoặc gần như không có cấu trúc, bỏ chúng khỏi chuỗi trước khi lấy mẫu đều; nếu sau khi loại bỏ quá nhiều frame mà video còn lại quá ngắn thì loại luôn video đó khỏi tập train.
- Đặt ngưỡng cho mỗi video: nếu hơn, ví dụ, 60% frame của một video bị phát hiện là hỏng, đánh dấu video đó là không đáng tin cậy và loại khỏi tập huấn luyện. Số lượng mẫu giảm nhẹ nhưng chất lượng dữ liệu hiệu dụng tăng lên.

Trùng lặp dữ liệu



Hình 14: Phân bố mức độ trùng lặp video trên bộ dữ liệu gốc.

Hình 14 cho thấy trong nhiều lớp, số lượng video bị phát hiện là trùng lặp chiếm một tỉ lệ đáng kể so với tổng số mẫu. Với một vài lớp ở phía bên trái biểu đồ, gần như một phần ba số video là bản sao của nhau.

Nếu không xử lý, các bản duplicate khiến mô hình tưởng rằng đang học từ rất nhiều ví dụ, trong khi thực tế chỉ xem đi xem lại cùng một chuỗi pixel. Điều này càng bất lợi khi tổng dữ liệu đã ít và phân bố nhãn mất cân bằng. Từ đó có thể đặt ra một quy tắc đơn giản:

- Giữ lại một bản duy nhất cho mỗi cụm duplicate, coi đó như một mẫu gốc.
- Với các lớp thuộc nhóm Tail vốn đã ít dữ liệu, có thể giữ lại thêm một số bản copy nhưng áp dụng augmentation mạnh hơn, thay vì giữ nguyên bản giống hệt nhau.

Tóm lại, những biểu đồ EDA bổ sung này cho thấy lượng dữ liệu hiệu dụng thực sự có thể nhỏ hơn tưởng tượng do video lỗi và video trùng. Đây là động lực để thử thêm hai hướng mở rộng: (i) dùng các kiến trúc mô hình tận dụng pretraining mạnh hơn, và (ii) điều chỉnh hàm mất mát để đối phó tốt hơn với phân bố nhãn đuôi dài.

VIII.2. Đổi backbone sang Vision Transformer (ViT)

Một hướng mở rộng về kiến trúc là thay ConvNeXt bằng Vision Transformer (ViT). Với bài toán nhận diện ký hiệu tay, việc nắm bắt mối quan hệ không gian tinh vi giữa các vùng trong ảnh (bàn tay, khuôn mặt, vai, ngực, ...) rất quan trọng. ViT là ứng viên phù hợp vì chia ảnh thành các patch nhỏ, biến mỗi patch thành một token và xử lý toàn bộ chuỗi token bằng Transformer.

Với mỗi frame, ta thu được một chuỗi patch embedding; mô hình có thể học trực tiếp các quan hệ như “bàn tay tương quan với khuôn mặt ra sao”, “cánh tay nằm ở vị trí nào so với thân người”, mà không bị giới hạn bởi receptive field cục bộ.

Việc thay ConvNeXt bằng ViT trong kiến trúc hiện tại khá tự nhiên. Có thể định nghĩa một backbone ViT như sau:

Thay backbone ConvNeXt bằng ViT

```

1 import timm
2 import torch.nn as nn
3
4 class ViTBackbone(nn.Module):
5     def __init__(self, model_name="vit_base_patch16_224", pretrained=True):
6         super().__init__()
7         self.vit = timm.create_model(
8             model_name,
9             pretrained=pretrained,
10            num_classes=0,    # bỏ classifier head
11            global_pool="avg" # lấy pooled embedding
12        )
13        self.out_dim = self.vit.num_features
14
15    def forward(self, x):
16        # x: (B*T, C, H, W)
17        return self.vit(x)    # (B*T, out_dim)

```

Phần còn lại của mô hình có thể giữ nguyên ý tưởng:

- Reshape đầu ra từ kích thước (B*T, D) về (B, T, D).
- Cộng thêm positional encoding theo thời gian.
- Đưa vào Transformer Encoder hoặc dùng Attention Pooling theo thời gian như đã trình bày.

ViT thường cần nhiều dữ liệu hơn CNN nếu huấn luyện từ đầu, nhưng trong bối cảnh này có thể tận dụng các mô hình ViT được pretrained rất mạnh (ImageNet-21K hoặc các tập lớn hơn). Một vài mẹo thực nghiệm hữu ích:

- Đóng băng một phần các layer đầu trong vài epoch đầu, chỉ fine-tune các lớp cuối.
- Sử dụng learning rate nhỏ cho các tham số pretrained, lớn hơn một chút cho các lớp mới thêm như temporal encoder hoặc classifier.
- Kết hợp augmentation mạnh và các kỹ thuật regularization như label smoothing, weight decay, dropout để giảm nguy cơ overfitting.

Với các cử chỉ khác nhau chỉ ở chi tiết nhỏ của bàn tay, backbone ViT có thể giúp mô hình phân biệt tốt hơn nhờ khả năng chú ý toàn cục trên không gian ảnh.

VIII.3. Thay đổi hàm Loss

Ngay cả khi đã áp dụng balanced sampler, bài toán vẫn chịu ảnh hưởng mạnh của phân phối đuôi dài: nhiều lớp chỉ có vài chục, thậm chí dưới 10 video. Một hướng mở rộng hợp lý là thay đổi hàm mất mát để mô hình quan tâm hơn đến các lớp hiếm.

Weighted Cross-Entropy

Ý tưởng là gán cho mỗi lớp một trọng số w_c tỉ lệ nghịch với số lượng mẫu của lớp đó. Khi tính Cross-Entropy, lỗi của lớp hiếm sẽ được khuếch đại hơn lỗi của lớp phổ biến.

Hàm mất mát có dạng:

$$\mathcal{L}_{\text{WCE}} = - \sum_{c=1}^C w_c y_c \log p_c,$$

trong đó y_c là nhãn one-hot, p_c là xác suất mô hình dự đoán cho lớp c , w_c có thể được chọn là

$$w_c = \frac{1}{\text{count}_c + \epsilon},$$

rồi chuẩn hoá lại sao cho $\sum_c w_c = C$.

Trong PyTorch, cách cài đặt khá gọn:

Weighted Cross-Entropy trong PyTorch

```

1 import torch.nn as nn
2 import numpy as np
3 import torch
4
5 # class_counts: mảng numpy kích thước [num_classes]
6 class_weights = 1.0 / (class_counts + 1e-6)
7 class_weights = class_weights / class_weights.mean() # chuẩn hoá
8 class_weights = torch.FloatTensor(class_weights).to(device)
9
10 criterion = nn.CrossEntropyLoss(
11     weight=class_weights,
12     label_smoothing=0.1
13 )

```

Khi kết hợp với balanced sampler, Weighted Cross-Entropy giúp mô hình “nghe” tiếng nói của các lớp hiếm rõ hơn, trong khi vẫn giữ được sự ổn định trên các lớp phổ biến.

Focal Loss và Weighted Focal Loss

Focal Loss được đề xuất trong bài toán phát hiện vật thể để xử lý mất cân bằng giữa foreground và background. Ý tưởng là giảm bớt ảnh hưởng của các mẫu dễ (đã dự đoán đúng với xác suất cao), tập trung gradient vào các mẫu khó.

Với một mẫu có nhãn đúng t và xác suất dự đoán p_t , Focal Loss có dạng:

$$\mathcal{L}_{\text{focal}} = -(1 - p_t)^\gamma \log(p_t),$$

trong đó $\gamma \geq 0$ là tham số điều khiển mức độ tập trung. Khi p_t gần 1, hệ số $(1 - p_t)^\gamma$ rất nhỏ, làm giảm đáng kể loss của mẫu đó; khi p_t còn thấp, hệ số này gần 1 và loss xấp xỉ Cross-Entropy bình thường.

Để xử lý đồng thời mất cân bằng giữa các lớp, có thể đưa thêm trọng số theo lớp α_c , thu được Weighted Focal Loss:

$$\mathcal{L}_{\text{WFocal}} = -\alpha_t(1 - p_t)^\gamma \log(p_t),$$

trong đó α_t là trọng số tương ứng với lớp đúng t .

Một hiện thực đơn giản trong PyTorch:

Weighted Focal Loss

```

1 import torch.nn.functional as F
2 import torch.nn as nn
3 import torch
4
5 class WeightedFocalLoss(nn.Module):
6     def __init__(self, class_weights=None, gamma=2.0):
7         super().__init__()
8         self.gamma = gamma
9         if class_weights is not None:
10             self.class_weights = class_weights
11         else:
12             self.class_weights = None
13
14     def forward(self, logits, targets):
15         # logits: (B, C), targets: (B,)
16         log_probs = F.log_softmax(logits, dim=-1)
17         probs = torch.exp(log_probs) # (B, C)
18
19         # lấy p_t và log(p_t) tương ứng với từng mẫu
20         targets = targets.view(-1, 1)
21         log_p_t = log_probs.gather(1, targets).squeeze(1)
22         p_t = probs.gather(1, targets).squeeze(1)
23
24         focal_factor = (1 - p_t) ** self.gamma
25
26         if self.class_weights is not None:
27             alpha_t = self.class_weights[targets.squeeze(1)]
28         else:
29             alpha_t = 1.0
30
31         loss = - alpha_t * focal_factor * log_p_t
32         return loss.mean()

```

Trong bối cảnh bộ dữ liệu ký hiệu tay:

- Các lớp Head thường được mô hình học tốt sau vài epoch; với Focal Loss, các mẫu thuộc những lớp này sẽ dần nhẹ tay hơn trong hàm mất mát, tránh chi phối quá mạnh.
- Các lớp Tail với ít dữ liệu và khó phân biệt hơn sẽ liên tục tạo ra loss lớn hơn, buộc mô hình dành thêm năng lực biểu diễn để tách biệt chúng khỏi các lớp còn lại.

Kết hợp Weighted Focal Loss với balanced sampler và backbone mạnh như ViT hoặc ConvNeXt là một hướng thử nghiệm hứa hẹn cho những bài toán “ít dữ liệu, nhiều lớp, mất cân bằng” như nhận diện ngôn ngữ ký hiệu trong cuộc thi này.

Tổng kết lại, các phân tích EDA mở rộng gợi ý rằng dữ liệu không chỉ ít và mất cân bằng, mà còn chứa một lượng đáng kể video lỗi và trùng lặp. Từ đó, có thể tiếp tục thử nghiệm các bước làm sạch dữ liệu sâu hơn, sử dụng backbone pretrain mạnh như ViT, và điều chỉnh hàm mất mát (Weighted Cross-Entropy, Weighted Focal Loss) để mô hình đối xử công bằng hơn với các lớp hiếm. Nếu xem pipeline ban đầu là phiên bản ổn định, thì những phương pháp mở rộng này là các bước tiếp theo để đẩy mô hình tiến gần hơn tới giới hạn mà bộ dữ liệu hiện có cho phép.

Phụ lục

1. **Code:** Các file code được đề cập trong bài có thể được tải tại [đây](#).
2. **Dataset:** Bộ dữ liệu có thể được tải tại [đây](#).
3. **Contest:** Thể lệ về cuộc thi có thể đọc tại [đây](#).
4. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng 3-4 tiếng.

AIO_QAs-Verified

🔒 Private group · 1.4K members



Hình 15: Hình ảnh group facebook AIO Q&A.